claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_a2b97cff-dcd\agent-a27cec2c50c2cc098.jsonl`
- SHA-256: `f11ce3a46b96eea56468f793f9d20b556c26ba0443d8bc21c773f456eb20bfc5`
- Source modified: `2026-06-21T13:55:40+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_a2b97cff-dcd`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`

Transcript

1. user (2026-06-21T13:52:09.108Z)

Research how to DISTRIBUTE a Python app so a non-technical user downloads and runs it WITHOUT sourcing the repo. Compare: (a) PyPI wheel + pip, (b) pipx, (c) uv tool / uvx, (d) private index / direct wheel download, (e) offline/air-gapped install bundles. For our app (console script `indexer`, FastAPI server + worker, optional torch via --index-url, requires-python>=3.10). Cover: console_scripts launch UX, how torch's --index-url need is handled across these channels, Windows-first user reality, offline install. Use web search for current best practices (2024-2026).

Return the RESEARCH schema. Context about our app:

PROJECT: a local, AI-powered badminton/racket-sports HIGHLIGHT INDEXER + rally detector.
Engine repo (PRIMARY): C:/Users/avidu/Projects/badminton-highlight-indexer (Python, FastAPI + SQLite + ffmpeg + GPU CV/AI; paid Gemini = cloud AI).
Sibling repos: C:/Users/avidu/Projects/khelsutra (the web/ React SPA + site/ marketing), C:/Users/avidu/Projects/rally-annotator (VLC plugin, MIT), C:/Users/avidu/Projects/sports-data-collector, C:/Users/avidu/Projects/sports-obsreport.

GOAL of the plan we are building: bake the engine into a DOWNLOADABLE, "batteries-included" Python APP that a user can download and run WITHOUT sourcing the repo (no git clone / build-from-source). It must spin up a local WEB SERVICE that runs (a) INFERENCE (video -> rallies -> highlight reel), (b) TRAINING (BYO model on their own labelled data), and (c) RALLY ANNOTATION if needed -- runnable BOTH locally and against CLOUD services. Preserve every already-completed end-to-end CUJ.

KEY FACTS the orchestrator already verified:
- pyproject.toml: hatchling build, namespace package (no backend/__init__.py), packages=["backend","training"], console script: indexer = backend.main:main. requires-python >=3.10.
- torch/torchvision are DELIBERATELY NOT in dependencies -- their CUDA/CPU wheels need --index-url (PEP621 cannot express). Optional extras: gpu(empty, docs-only), auth(PyJWT), reporting(git+ssh obsreport), storage-s3(boto3), storage-gcs(google-cloud-storage). google-genai is a hard dep.
- Server today: python -m backend.main --server --port 8000 . Worker: python -m backend.worker {infer|train} . CLI single-shot: python -m backend.main --video-path ... --segmenter {gemini|yolo_hybrid|wasb_hybrid|motion}.
- THE HARD PACKAGING PROBLEM: native WASB shuttle model runs in a SEPARATE Python env (conda, torch 1.11+cu113) while the main engine uses torch 2.6+cu124. ffmpeg/ffprobe are system deps. Model weights (wasb_badminton_best.pth.tar) live in a GCS bucket.
- Decoupled-compute SEAM already built (M1-M8 merged, default-OFF): deployment.profile + compute_target on AppConfig; backend/config/presets.py (PROFILE_PRESETS: truly-local / cloud-serving / cpu-mock); backend/storage/* (local/gcs/s3/gdrive StorageBackend registry); backend/compute/* (ComputeBackend registry, CloudRunCompute dispatches a Cloud Run Job); backend/config/startup.py per-profile checks; backend/billing.py cost ledger.
- The web SPA lives in a DIFFERENT repo (khelsutra/web, Vite/React/TS/Tailwind, i18n en+kn+hi).

The engine has a likely-DEAD backend/api/static. Bundling the UI is a cross-repo concern.
- Licensing guardrail (NON-NEGOTIABLE): every dependency -- code, data, AND weights -- must be MIT/Apache-2.0/BSD only. Paid LLMs = inference/serving only, never training data. Any NEW packaging dependency must also be license-clean.
- A prior docs/DESKTOP_APP_PLAN.md exists but was SUPERSEDED by owner decision D10: deliver a "self-hosted webserver + UI the user spins up" (NOT a desktop app).
khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md is the live appliance design.
Always ground claims in file:line or a doc path. Read the actual files; do not assume.

2. assistant (2026-06-21T13:52:12.194Z)

I'll research this thoroughly. Let me start by reading the actual project files to ground my claims, then do web research on current packaging best practices.

3. user (2026-06-21T13:52:13.147Z)

```
1       # PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
2       #
3       # Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4       # (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
5       # reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6       # auto-detection would otherwise miss the un-__init__'d `backend` root.
7       #
8       # torch/torchvision are deliberately NOT listed in [project.dependencies]:
9       # their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
10      # which PEP 621 cannot express. Install them separately ? see [project.optional-
dependencies]
11      # `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12      [build-system]
13      requires = ["hatchling"]
14      build-backend = "hatchling.build"
15
16      [project]
17      name = "badminton-highlight-indexer"
18      version = "0.1.0"
19      description = "AI-assisted rally segmentation and highlight indexing for racket sports."
20      readme = "README.md"
21      requires-python = ">=3.10"
22      license = { text = "MIT" }
23      authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
24      keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
25
26      # Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
27      dependencies = [
28          "fastapi>=0.111.0",
29          "uvicorn>=0.30.1",
30          "opencv-python>=4.9.0.80",
31          "numpy>=1.26.4",
32          "pydantic>=2.7.1",
33          "jinja2>=3.1.4",
34          "python-dotenv>=1.0.0",
35          "google-genai>=2.4.0",
36          "supervision>=0.21.0,<0.30.0",
37      ]
38
39      [project.optional-dependencies]
40      # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
41      dev = [
42          "pytest",
43          "pytest-cov",
44          "httpx",        # fastapi TestClient transport
45          "ruff",
46          "mypy",
47      ]
48
```

```
49      # Per-video observability reports (SOFT dependency: the pipeline degrades to a
50      # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
51      # fetched over SSH; if you lack access, simply omit this group ? reporting
52      # tests skip via pytest.importorskip. For local dev against a sibling checkout:
53      #   pip install -e ../sports-obsreport --no-build-isolation
54      reporting = [
55          "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
56      ]
57
58      # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
59      # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
60      # express, so they are NOT declarable here. Install them out-of-band instead:
61      #   # CUDA 12.4 box:
62      #   pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
63      #   # CPU-only box (also what CI uses):
64      #   pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
65      gpu = []
66
67      # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
68      # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
69      # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
R2 +
70      # DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally
71      # MOUNTED Google Drive (Google Drive for Desktop) as plain files; see
backend/storage/gdrive.py.
72      storage-s3 = ["boto3>=1.34"]
73      storage-gcs = ["google-cloud-storage>=2.16"]
74
75      [project.scripts]
76      # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
77      indexer = "backend.main:main"
78
79      [tool.hatch.metadata]
80      # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
81      # reference*); hatchling requires opting in to allow those in metadata.
82      allow-direct-references = true
83
84      [tool.hatch.build.targets.wheel]
85      # `backend` is a namespace package (no __init__.py); list both import roots
86      # explicitly so an editable/wheel install exposes `import backend` and
87      # `import training` regardless of cwd.
88      packages = ["backend", "training"]
89
```

4. assistant (2026-06-21T13:52:16.378Z)

Now let me read the key engine files referenced in the task to ground my claims, while kicking off web research in parallel.

5. user (2026-06-21T13:52:17.873Z)

```
1       import os
2       import sys
3       import argparse
4       import logging
5       import uuid
6       from urllib.parse import quote
7       import uvicorn
8       from dotenv import load_dotenv
9
10      load_dotenv()
11
12      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
```

```
    logging.basicConfig(
        level=logging.INFO,
        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
        datefmt='%H:%M:%S'
    )
    logger = logging.getLogger(__name__)
    from fastapi import FastAPI, HTTPException
    from fastapi.middleware.cors import CORSMiddleware
    from pydantic import BaseModel
    from typing import List, Dict, Any, Optional

    from fastapi.staticfiles import StaticFiles
    from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
    from backend.pipeline.validators.base import registry
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.stitcher.compiler import VideoCompiler
    from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
    from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
    from backend.config import (  # new typed config
        load_config as load_app_config,
        AppConfig,
        StitchingConfig,
        enforce_startup_checks,
        StartupCheckError,
    )
    from backend.results import Failure  # standardized error/result contract
    from backend.reporting import emit_indexer_report
    from backend import storage  # M2: URI-aware input acquisition (local = identity
passthrough)
    from backend import billing  # M8: per-job cost ledger (USD internal / INR display)

    app = FastAPI(title="Sports Highlight Indexer API")

    # Enable CORS for the local web interface. allow_origins='*' with allow_credentials=True
is
    # rejected by browsers per the fetch spec and is an unsafe default to carry into the
    # auth/tenancy work; this tool uses no cookies, so credentials stay off.
    app.add_middleware(
        CORSMiddleware,
        allow_origins=["*"],
        allow_credentials=False,
        allow_methods=["*"],
        allow_headers=["*"],
    )

    # Serves static frontend files directly (now under api/ per approved structure)
    os.makedirs("backend/api/static", exist_ok=True)
    app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")

    @app.get("/", response_class=HTMLResponse)
    def serve_index():
        index_path = "backend/api/static/index.html"
        if os.path.exists(index_path):
            with open(index_path, "r", encoding="utf-8") as f:
                return f.read()
        return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"

    # Global variables initialized at startup
    db_instance: Optional[Database] = None
    global_config: Optional[AppConfig] = None  # now typed (Pydantic model)

    # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
```

P3)
74   # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
75   # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
76   # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
77   # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
78   # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
79   # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
80   from backend.api.job_worker import (# noqa: F401 (re-exports: names stay addressable
on backend.main)

6. user (2026-06-21T13:52:24.712Z)

Web search results for query: "pip install torch --index-url cannot be expressed in
pyproject.toml dependency 2025"

Links: [{"title":"dependencies - A Developer's Guide to Portable PyTorch Installation with uv
and pip - portability","url":"https://iifx.dev/en/articles/460177044/a-developer-s-guide-to-
portable-pytorch-installation-with-uv-and-pip"},{"title":"missing --index-url in `uv pip compile
pyproject.toml` for packages that come from `explicit = true` indexes · Issue #15534 · astral-
sh/uv","url":"https://github.com/astral-sh/uv/issues/15534"},{"title":"Installing PyTorch: An
index URL could not be queried · Issue #8344 · astral-sh/uv","url":"https://github.com/astral-
sh/uv/issues/8344"},{"title":"Cannot install, torch does not exist and pytorch says I must
install torch · Issue #113787 ·
pytorch/pytorch","url":"https://github.com/pytorch/pytorch/issues/113787"},{"title":"Add
--extra-index-url option to pyproject.toml to install requirements.txt · Issue #13000 ·
pypa/pip","url":"https://github.com/pypa/pip/issues/13000"},{"title":"How to use different index
url with `pyproject.toml` when using `uv pip compile` · Issue #4173 · astral-
sh/uv","url":"https://github.com/astral-sh/uv/issues/4173"}]

Based on the search results, I found relevant information about this issue:

The core problem is that PyTorch, unlike most Python libraries, distributes its platform-
specific builds on separate index URLs, not just the main Python Package Index (PyPI). This is
why the standard pip `--index-url` flag cannot be directly expressed in `pyproject.toml`
dependency specifications.

There are several approaches to handle this:

1. **Using uv with tool.uv.sources**: A cleaner approach with uv is to define platform-specific
indexes using environment markers directly in your pyproject.toml, which allows you to define
different install behaviors based on the user's operating system or other conditions.

2. **Using the @ syntax**: You can try adding it to dependencies using syntax like
`"torch==2.3.1+cpu @ https://download.pytorch.org/whl/cpu"`, though this approach has
limitations.

3. **Manual pip installation**: The `--no-deps torch` flag tells pip to ignore the 'torch'
requirement in any package's metadata, which is often the most reliable approach for pure pip
users.

The fundamental limitation is that standard PEP 508 dependency specifications don't natively
support custom index URLs, so alternative solutions like using `uv` with its `tool.uv.sources`
configuration or manual installation workarounds are typically necessary.

REMINDER: You MUST include the sources above in your response to the user using markdown
hyperlinks.

7. user (2026-06-21T13:52:28.551Z)

Web search results for query: "pipx vs uv tool install Python CLI app distribution 2025 best
practices"

Links: [{"title":"How to choose between uvx and pipx for Python CLI tools in 2026 |

BSWEN","url":"https://docs.bswen.com/blog/2026-03-05-uvx-vs-pipx/"},{"title":"Comparisons - pipx","url":"https://pipx.pypa.io/stable/explanation/comparisons/"},{"title":"How do uv tool and pipx compare? | pydevtools","url":"https://pydevtools.com/handbook/explanation/how-do-uv-tool-and-pipx-compare/"},{"title":"pipx: Install Python CLI Tools in Isolation | pydevtools","url":"https://pydevtools.com/handbook/reference/pipx/"},{"title":"uv: A Complete Guide to Python's Fastest Package Manager | pydevtools","url":"https://pydevtools.com/handbook/explanation/uv-complete-guide/"},{"title":"uv: The Python Tool That Should Have Existed Ten Years Ago | by Simardeep Singh | Medium","url":"https://medium.com/@simardeep.oberoi/uv-the-python-tool-that-should-have-existed-ten-years-ago-46ff611195ca"},{"title":"Ultimate guide to uv library in Python","url":"https://deepnote.com/blog/ultimate-guide-to-uv-library-in-python"},{"title":"Switching from Pip to uv in Python: A Comprehensive Guide | AppSignal Blog","url":"https://blog.appsignal.com/2025/09/24/switching-from-pip-to-uv-in-python-a-comprehensive-guide.html"},{"title":"pipx Backend | mise-en-place","url":"https://mise.jdx.dev/dev-tools/backends/pipx.html"}]

I found comprehensive information about comparing pipx and uv for Python CLI app distribution. Here are the key findings from 2025-2026 sources:

Key Differences

Speed & Performance:
uv is 10-100x faster than pipx and offers better features like Python version control, plugin support, and automatic PATH configuration. uv tool install ruff finishes in roughly the time a hot uvx ruff invocation takes, while pipx with its default pip backend takes several seconds for the same install because it shells out to pip and virtualenv.

Core Functionality:
Both uv tool and pipx install a Python tool into its own virtual environment and expose the tool's console scripts on PATH. Both have a one-shot run mode (uvx and pipx run). The key difference: uvx runs tools ephemerally (no installation), while uv tool install creates permanent executables.

Python Version Management:
Both uvx and uv tool install support the --python flag: if the specified Python version isn't installed, uv downloads it automatically. In contrast, pipx uses whichever Python is already on the system.

PATH Configuration:
pipx requires adding ~/.local/bin/ to PATH. Many beginners struggle with this. uv handles PATH configuration automatically.

Best Practices for 2025

For New Projects:
For a new setup, install uv and use uv tool and uvx. One binary covers Python installs, project work, scripts, and CLI tools, and the install speed is consistently faster.

For Existing pipx Users:
pipx install pipx[uv] keeps every pipx command intact but switches the venv-creation step to uv. Existing scripts and CI that call pipx install keep working; cold installs get faster.

When to Stay with pipx:
Stay on pipx when a workflow depends on inject / uninject, --suffix, --global, install-all, or man-page integration.

Usage Patterns:
The distinction between uvx and uv tool install maps to how you use the tool. If you run it occasionally or want to try it once, use uvx. If you use it daily and want it on your PATH, use uv tool install.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

8. assistant (2026-06-21T13:52:31.328Z)

Let me read more engine files and continue research in parallel.

9. user (2026-06-21T13:52:33.548Z)

e68b583 fix(api): resolve a bucket-ingested video's source to a local file before stitching (#289)
43eccc1 feat(serving): M8 ? per-job cost ledger + versioned pricebook + /cost API (default-OFF) (#288)
8cdf65c feat(serving): M6 ? Cloud Run GPU dispatch shim + worker image (default-OFF, mocked) (#287)
---WORKER---

10. user (2026-06-21T13:52:39.818Z)

Web search results for query: "uv tool install PyTorch index pyproject.toml tool.uv.index explicit 2025"

Links: [{"title":"Using uv with PyTorch | uv","url":"https://docs.astral.sh/uv/guides/integration/pytorch/"},{"title":"How to Install PyTorch with uv | pydevtools","url":"https://pydevtools.com/handbook/how-to/how-to-install-pytorch-with-uv/"},{"title":"Package indexes | uv","url":"https://docs.astral.sh/uv/concepts/indexes/"},{"title":"Managing dependencies | uv","url":"https://docs.astral.sh/uv/concepts/projects/dependencies/"},{"title":"Project Setup for Machine Learning with PyTorch and UV | by Florian Trautweiler | Medium","url":"https://medium.com/@florian-trautweiler/project-setup-for-machine-learning-with-pytorch-and-uv-8e3bbb37eb5e"},{"title":"Installation ? Deep Learning","url":"https://perso.esiee.fr/~chierchg/deep-learning/README.html"},{"title":"Useful Features of the Python Project Management Tool uv up to v0.5.3 ? Dependencies Edition | by Turing Inc. | Medium","url":"https://medium.com/@turingmotors/5-useful-features-of-the-python-project-management-tool-uv-up-to-v0-5-3-dependencies-edition-9aab2cddceeb"},{"title":"uv/docs/configuration/indexes.md at c58675fdac6bdc16a3c0ab4090600282d49db369 - mirrors/uv - Forgejo: Beyond coding. We Forge.","url":"https://git.joshthomas.dev/mirrors/uv/src/commit/c58675fdac6bdc16a3c0ab4090600282d49db369/docs/configuration/indexes.md"}]

Based on the search results, here's information about using `uv` to install PyTorch with index configuration in `pyproject.toml`:

Key Configuration Pattern

To add the relevant PyTorch index to your pyproject.toml, you can define `[[tool.uv.index]]` entries with `explicit = true`, such as:

```toml
[[tool.uv.index]]
name = "pytorch-cpu"
url = "https://download.pytorch.org/whl/cpu"
explicit = true
```

Why Use explicit = true

The recommendation is to use `explicit = true` to ensure that the index is only used for torch, torchvision, and other PyTorch-related packages, as opposed to generic dependencies like jinja2, which should continue to be sourced from the default index (PyPI).

Routing Packages to the Index

Update the pyproject.toml to point torch and torchvision to the desired index using `[tool.uv.sources]`:

```toml
[tool.uv.sources]
torch = [{ index = "pytorch-cpu" }]
torchvision = [{ index = "pytorch-cpu" }]
```

Platform-Specific Configuration

You can gate on sys_platform to instruct uv to use the PyTorch index on Linux and Windows, but fall back to PyPI on macOS:

```toml
[tool.uv.sources]
torch = [{ index = "pytorch-cu118", marker = "sys_platform == 'linux' or sys_platform == 'win32'" }]
```

The official Astral documentation provides comprehensive guides on this configuration pattern for various CUDA versions and platforms.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

11. user (2026-06-21T13:52:40.731Z)

Web search results for query: "distribute Python app to non-technical Windows user without git clone PyInstaller vs pip 2025"

Links: [{"title":"Using PyInstaller to Easily Distribute Python Applications ? Real Python","url":"https://realpython.com/pyinstaller-python/"},{"title":"Using PyInstaller ? PyInstaller 4.1 documentation","url":"https://pyinstaller.org/en/v4.1/usage.html"},{"title":"Python Program Distribution and Installation Methods | Server As Code Dot Com","url":"https://serverascode.com/2024/12/21/python-installation-methods.html"},{"title":"pyinstaller · PyPI","url":"https://pypi.org/project/pyinstaller/"},{"title":"PyInstaller: Bundle a Python Application Into a Single Executable | CodeCut","url":"https://codecut.ai/pyinstaller-bundle-a-python-application-into-a-single-executable/"},{"title":"Using PyInstaller to Easily Distribute Python Applications|Documentation","url":"https://docs.aspose.com/cells/python-java/pyinstaller-python/"}]

Based on the search results, here's what I found about distributing Python apps to non-technical Windows users using PyInstaller vs pip:

PyInstaller Approach

PyInstaller gives you the ability to create a folder or executable that users can immediately run without any extra installation. This is the key advantage for non-technical users. PyInstaller bundles a Python application and all its dependencies into a single package, and the user can run the packaged app without installing a Python interpreter or any modules.

The process works by having PyInstaller read your Python script, analyze your code to discover every module and library needed, then collect copies of all those files including the active Python interpreter and put them with your script in a single folder, or optionally in a single executable file.

Key advantages: PyInstaller bundles a Python application and all its dependencies into a single package, and the user can run the packaged app without installing a Python interpreter or any modules.

Pip Approach

Using pip for distribution is generally not suitable for non-technical Windows users because distributing Python scripts that require installing Python, managing dependencies, and setting up environments creates barriers for end-users. Pip requires users to have Python installed and understand virtual environments and dependency management.

Recommendation

For non-technical Windows users, **PyInstaller is clearly the better choice**. It eliminates the need for users to have Python installed or understand command-line tools. They simply download and run the executable?making it much more user-friendly than requiring pip installation with all its prerequisite knowledge.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

12. assistant (2026-06-21T13:52:43.564Z)

Let me read the worker.py and the appliance architecture doc, plus fetch the authoritative uv PyTorch guide.

13. user (2026-06-21T13:52:45.033Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer. Did you mean worker?

14. user (2026-06-21T13:52:54.064Z)

uv PyTorch Installation with Custom Indexes

Project-Based Approach (Primary Method)

uv handles PyTorch installation through **`pyproject.toml` configuration only** ? this works for project-based workflows (`uv lock`, `uv sync`, `uv run`), not for `uv tool install` or `uvx`.

Key Configuration Components

1. `tool.uv.index` ? Define Custom PyTorch Indexes

Register PyTorch's specialized indexes with `explicit = true` to ensure they're used only for PyTorch packages:

```toml
[[tool.uv.index]]
name = "pytorch-cpu"
url = "https://download.pytorch.org/whl/cpu"
explicit = true
```

2. `tool.uv.sources` ? Route Packages to Specific Indexes

Direct torch and torchvision to the appropriate index using markers or extras:

```toml
[tool.uv.sources]
torch = [
  { index = "pytorch-cpu", marker = "sys_platform != 'linux'" },
  { index = "pytorch-cu130", marker = "sys_platform == 'linux'" },
]
```

3. Platform Markers

Use environment markers like `sys_platform` to select backends per OS. This enables "CPU-only on macOS/Windows, CUDA on Linux" strategies or ROCm/Intel GPU targeting.

4. Optional Dependencies

Use extras to allow users to choose backends: `uv sync --extra cpu` vs. `uv sync --extra cu130`, with conflict declarations preventing simultaneous selection.

`--torch-backend` Flag (pip Interface Only)

Available exclusively in `uv pip` workflows:

- `--torch-backend=auto`: Automatically detects installed GPU drivers and selects the compatible PyTorch index
- `--torch-backend=cu130`: Explicitly specifies a CUDA 13.0 backend

This flag does **not** apply to `uv tool install` or `uvx`.

15. user (2026-06-21T13:52:55.950Z)

Web search results for query: "pip download offline air-gapped wheels --platform --python-version install no internet 2025"

Links: [{"title":"Air-gapped Python: Setting up Python without a net(work) | InfoWorld","url":"https://www.infoworld.com/article/3836692/airgapped-python-setting-up-python-without-a-network.html"},{"title":"pip download - pip documentation v26.1.2","url":"https://pip.pypa.io/en/stable/cli/pip_download/"},{"title":"How to Install Python Wheel Dependencies from a Local Folder with pip: A Poor Man's Private PyPI Guide","url":"https://www.py4u.org/blog/python-pip-install-wheel-dependencies-from-a-folder/"},{"title":"Pip download wheel without install","url":"https://codegive.com/blog/pip_download_wheel_without_install.php"},{"title":"Enterprise / Air-Gapped Installation | Spec Kit Documentation","url":"https://github.github.com/spec-kit/install/air-gapped.html"},{"title":"pip download ? pip 10.0.0.dev0 documentation","url":"https://pip-python3.readthedocs.io/en/latest/reference/pip_download.html"},{"title":"Installing python packages in air-gapped environment. | by Kriyanshishah | Medium","url":"https://medium.com/@kriyanshii/installing-python-packages-in-air-gapped-environment-7c9bfddff2b0"},{"title":"Airgapped Python: Setting up Python without a net(work) - Azalio","url":"https://www.azalio.io/airgapped-python-setting-up-python-without-a-network/"},{"title":"Install Python Libraries in an air-gapped environment","url":"https://www.linkedin.com/pulse/install-python-libraries-air-gapped-environment-sif-baksh"},{"title":"pip download ? pip 9.1.0.dev0 documentation","url":"https://readdoctest.readthedocs.io/en/latest/reference/pip_download.html"}]

Based on the search results, here's comprehensive information about using `pip download` with `--platform` and `--python-version` for offline, air-gapped wheel installations:

Key Capabilities

pip download with the --platform, --python-version, --implementation, and --abi options provides the ability to fetch dependencies for an interpreter and system other than the ones that pip is running on.

Important Requirements

--only-binary=:all: or --no-deps is required when using any of these options. Additionally, to avoid fetching dependencies that happen to match the constraint of the current interpreter (but not your target one), it is recommended to specify all of these options if you are specifying one of them.

Python Version Specification

You can pass --python-version <version_number>. This flags pip download to specify the version of Python so you obtain wheels for that version. The version can be specified using up to three

dot-separated integers (e.g. "3" for 3.0.0, "3.7" for 3.7.0, or "3.7.3").

Offline Installation Process

Once you've downloaded wheels, create a virtual environment for the app as you usually would, then run pip: pip install <requirements> --no-index -f /path/to/wheels, where --no-index forces pip to ignore checking PyPI for packages. The -f option lets you provide a path for pip to search for Python .whl files.

Best Practices

With pip download, keep in mind that you should always run pip from the same version of Python you intend to use on the target machine. If, for instance, you're using Python 3.12 on the air-gapped machine, you'll need to run pip download using Python 3.12 on the networked machine.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

16. assistant (2026-06-21T13:52:58.348Z)

The worker file is named differently. Let me find it and read the appliance architecture doc plus presets.

17. user (2026-06-21T13:52:59.043Z)

No files found

18. user (2026-06-21T13:52:59.844Z)

C:/Users/avidu/Projects/khelsutra/docs\LOCAL_APPLIANCE_ARCHITECTURE.md

19. user (2026-06-21T13:53:00.130Z)

```
1       """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3       A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4       config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5       knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7       Precedence (lowest ? highest), realised by the loader::
8
9           built-in model defaults  <  profile preset  <  config.json  <  env  <  worker --set
10
11      This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12      module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13      (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14      profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15      today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16      spend is never entered silently).
17      """
18
19      from __future__ import annotations
20
21      import copy
22      import os
23      from typing import Any, Mapping, Optional
24
25      # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
```

```
26      # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27      PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28      COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30      # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31      # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32      COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33          "truly-local": "local_gpu",
34          "cloud-serving": "cloud_run",
35          "cpu-mock": "cpu_mock",
36      }
37
38      # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39      # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40      # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41      # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42      # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43      PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44          "truly-local": {
45              "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46              # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47              # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48              "indexing": {"skip_ai_handoff": True},
49              "storage": {"backend": "local"},
50          },
51          "cloud-serving": {
52              "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53              # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54              "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
55              "storage": {"backend": "gcs"},
56          },
57          "cpu-mock": {
58              "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
59              # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
60              "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
```

20. assistant (2026-06-21T13:53:02.951Z)

The worker is `backend/worker/` (a package). Let me find the entry and read the appliance doc which is the live design target.

21. user (2026-06-21T13:53:03.650Z)

```
backend\worker\__init__.py
backend\worker\__main__.py
```

22. user (2026-06-21T13:53:04.196Z)

```
1       # Single-box appliance ? UI-powered rally indexer + reel generator (KhelSutra Studio)
2
3       > **Status:** `DRAFT` ? owner-gated (auth choice, retention, cost ceiling, GPU host, P5
bridge) · **Owner:** Avi · **Created:** 2026-06-20 · **Last updated:** 2026-06-20
4       > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5       > **Tracking anchors:** §9 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
```

> **Relation:** peer-of `PER_RALLY_SELECTION.md` / `REQUEST_ACCESS_FORM.md` in this repo; **consumes** the engine's decoupled `StorageBackend` + `python -m backend.worker` seams and the FastAPI loop; **extends** the engine's `HOSTING_PLAN.md` gating + `UI_PROPOSAL.md` alpha screens. Engine-side changes here are **proposals to coordinate** with the active decoupled/worker sessions ? not written unilaterally.
> **Honesty note:** every engine-fact claim is tagged `[verified]` (read against engine source this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not legal/financial advice.

---

0. TL;DR

KhelSutra Studio turns the engine from a CLI into a **single-box, UI-powered rally indexer + reel generator**: the khelsutra `web/` React SPA, served behind an **edge auth gate**, drives the **existing** engine FastAPI loop (ingest ? index ? pick rallies ? reel ? download) on **one machine**, with `StorageBackend=local` for the MVP. It is the **single-box deployment profile of the just-shipped decoupled architecture** ? storage=local + compute=local is the degenerate collapse of the same storage/worker split.

The **single most important caveat:** the engine has **zero app-layer auth and CORS `*`** `[verified backend/main.py:42]`, so safety is structural **only** because the engine stays bound to `127.0.0.1:8000` `[verified backend/main.py:638]`, a firewall blocks inbound `8000`, and a reverse proxy/tunnel is the sole authenticated ingress ? **the gate, not the app, is the tenancy boundary.**

Two honest hardware truths: (1) the current droplet is **CPU-only** ? real detection there is **Gemini (cloud)** or **stub (CPU mock)**; **real local-GPU detection needs a GPU host** (native WASB, in-flight). (2) The owner priority is to **stand up a GPU host ASAP** ? provider-agnostic (DO GPU droplet / GCE GPU VM / Cloud Run+GPU) ? but that runs **in parallel** with the UI MVP, which ships on Gemini today.

1. Problem & goal

The decoupled engine just shipped a `StorageBackend` ABC + a `python -m backend.worker {infer|train}` dispatcher `[verified backend/storage/, backend/worker/__main__.py]`, and the FastAPI loop already covers `process ? jobs ? query ? compile ? highlights` `[verified backend/main.py]`. What's missing is a **console** that lets a non-CLI operator run the whole loop on a single box, **honestly, behind a gate**.

Headline CUJ (owner's words): a **hosted website** where I pick an input ? **a Google Drive file, a bucket file (`s3://`/`gs://`), or a local upload** ? it lands in the **bucket (DO Spaces / GCS)** or locally ? I **generate highlights** on a **GPU-enabled host** running the real e2e ? I **pick rallies, watch, and download** ? **completely from the UI.**

Outcome ("good"): an operator signs in, picks a source, watches it index, curates rallies, downloads a reel ? on the **live CPU droplet today** (Gemini/stub), and on a **GPU host later** (native WASB) with **no UI fork**.

Read to get current: `PER_RALLY_SELECTION.md` (§3/§4/§10 contract anchors), engine `HOSTING_PLAN.md` (gating), `DECOUPLED_COMPUTE/HANDOFF.md` (what's built), `VIDEO_LOCALITY_MODEL.md` (privacy/retention).

2. Decisions locked

#	Decision	Source / date	Implication
D1	Reverse-proxied **single origin**; engine stays `127.0.0.1:8000`	`[verified main.py:638]`; 2026-06-20	CORS `*` is moot from the browser ? the proxy is the boundary
D2	**Edge auth**, zero engine auth code	`HOSTING_PLAN.md`; 2026-06-20	Caddy basicauth OR cloudflared + Cloudflare Access
D3	MVP drives the **in-process FastAPI worker** loop, `storage=local`	`[verified job_worker.py:37, local.py:33]`; 2026-06-20	bucket worker = reserved scale path, out of MVP
D4	Embed **RallyPicker** from `PER_RALLY_SELECTION.md` **verbatim**	PR [#11]; 2026-06-20	the console is a superset; do not redesign the picker
D5	Console lives in the khelsutra **`web/`** React scaffold	`web/README.md`; 2026-06-20	Cloudflare-Pages-style root, not engine static
D6	Compute placement is a **setting**, rendered from `GET /api/capabilities`		

`[design]`; 2026-06-20 | UI offers ONLY what the box can honestly do |
36 | D7 | **Ingest is a source menu** (local upload · bucket URI · Drive) normalizing to one video object | `[verified backend/storage/ref.py parse_uri]`; 2026-06-20 | Drive is a **stub** today ? scoped honestly (§4.2) |
37 | D8 | GPU host is **provider-agnostic** (DO droplet / GCE VM / Cloud Run+GPU); storage stays the bucket | Owner 2026-06-20 | compute and storage are separate planes; no provider lock-in |
38 | D9 | **Observability is first-class** ? structured logs on every new path + a DoD item | Owner 2026-06-20 | see §5; verified during CUJ replay |
39 | D10 | Delivery = a **self-hosted webserver + UI the user spins up** (NOT a desktop app); the user points at **local storage + GPU/CPU OR cloud locations** and the UI drives the tool | Owner 2026-06-20 | **supersedes `DESKTOP_APP_PLAN.md`**; one artifact (this appliance) runs on the owner box, a droplet, or a GPU host ? same UI, compute/storage are settings |
40
41   ## 3. Foundation ? what the engine gives us today `[verified this session unless noted]`
42   - **FastAPI loop:** `POST /api/process` (202 + `job_id`) ? poll `GET /api/jobs/{id}` ? `POST /api/query` ? `POST /api/compile` ? `GET /api/highlights/{video_id}/{filename}` (FileResponse) `[main.py]`. **Chunked upload exists** ? `POST /api/upload/init` · `PUT /api/upload/{id}/part/{i}` · `POST /api/upload/{id}/complete` · `DELETE /api/upload/{id}` `[verified uploads.py:64,89,123,147]` ? but the bundled `app.js` **never calls them** (it only hits `/api/process` + `/api/compile` `[verified app.js:88,373]`). So browser upload is **client-wiring only**, no engine change.
43   - **In-process worker:** one module-global `ThreadPoolExecutor(max_workers=1)` `[verified job_worker.py:37]` ? strictly **one job at a time**; the DB `jobs` table is the clock; `fail_stale_jobs` sweeps queued/running on boot. Gemini chunk concurrency is a **separate** knob `indexing.ai_max_workers` (**default 5** `[verified models.py:183]`; set to 1 to serialize) ? it is **not** already 1.
44   - **Storage:** `storage.backend` default = `"local"` `[verified models.py:298]`; `LocalStorage.fetch_to_local` is **identity** `[verified local.py:33]`, `put` no-ops on `src==dst` ? so on a single box the worker's pull?pipeline?push **collapses to read-local/write-local**. URI scheme `parse_uri` `[verified ref.py]`: bare/`local://` ? local; `s3://` (AWS/R2/**DO Spaces**/MinIO, by endpoint) ; `gcs://`/`gs://` ; `gdrive://` ? **follow-up stub (emulated only)**. Credentials never in config ? boto3 default chain / `~/.aws` `[verified s3.py]`.
45   - **Compute seam:** `indexing.detector_impl` ? {`auto`=WSL, `native`=Linux-native, `stub`=CPU mock}; default `"auto"` `[verified models.py:129]`, resolved in `trajectory_hybrid._resolve_runner` `[verified :76-85]`. **Native WASB is in-flight on branch `feat/native-wasb-runner`:** `native_wasb_runner.py` **exists** `[verified]`; **`fusion_hybrid._build_native_runner` returns `NativeWasbRunner`** `[verified fusion_hybrid.py:86]` while **`trajectory_hybrid._build_native_runner` still raises** `[verified :57-63]`. Acceptance gate = **owner byte-parity** (WSL CSV == native CSV) on a real GPU box ? not exercisable on the droplet or in the offline suite. Treat native as **not production-verified**.
46   - **Worker CLI:** `python -m backend.worker {infer|train}` pulls a `--video-uri` ? pipeline ? pushes `outputs/<video_id>/{intervals.csv,report.json}` `[verified worker/__main__.py:88,149]`. The validated GPU-free/key-free combo: `detector_impl=stub` + `--set indexing.skip_ai_handoff=true` + `storage.s3`.
47   - **Gating:** cloudflared Tunnel (outbound-only, no inbound ports) + Cloudflare Access email-OTP; backend trusts `Cf-Access-Authenticated-User-Email` `[HOSTING_PLAN.md]`. `GET /api/health` **now EXISTS** (engine M1, [#268]) ? `{status, sport, default_segmenter}` `[verified main.py:127, corrected 2026-06-21 ? was "does not exist"]`.
48   - **Droplet reality:** `168.144.126.176`, **CPU-only, NO GPU**, region `blr1`; co-hosts the marketing site (`/srv/khelsutra-site`, node `:8787`) **and** the engine (`~/rally`); ran the full train+infer e2e against DO Spaces with the **GPU mocked** `[SETUP_NEW_MACHINE.md, DECOUPLED_COMPUTE/HANDOFF.md]`. Shared testbed: MinIO `9000/9001` + GCS emulator `9023` also run there.
49   - **Absent (net-new):** `GET /api/capabilities`, `GET /api/videos` (only single-row `get_video(id)`), `GET /api/reels/{video_id}`, any source/proxy streaming endpoint, any **Dockerfile**, any per-video **cost ledger** `[verified ? grep returns none]`.
50
51   ## 4. Design / architecture
52
53   ### 4.1 Topology & request path
54   ONE box: **edge gate** (cloudflared+Access or Caddy basicauth) ? **reverse proxy /

static host** serving the `web/` SPA bundle and proxying `/api/*` ? engine **FastAPI
`127.0.0.1:8000`** ? **LocalStorage** ? **detector seam** (Gemini/stub today; native WASB on a
GPU host). The firewall blocks inbound `8000` + the test ports `9000/9001/9023`. The marketing
site stays a **separate** systemd unit on a **separate** subdomain.

```
PUBLIC INTERNET
   ?  (1) EDGE GATE ? the ONLY ingress; auth lives HERE, not in the app
   ?      cloudflared Tunnel + Cloudflare Access (no inbound ports)  OR  Caddy :443 +
basicauth
   ?
????????????????????????????????????????????????????????????????????????????????
? ONE BOX   firewall BLOCKS inbound 8000 + 9000/9001/9023                    ?
? (2) reverse proxy / static host (Caddy/cloudflared)                        ?
?      /, /assets/*  ? web/ React SPA      /api/*, /api/highlights/* ? :8000 ?
?            ?  ONE ORIGIN to the browser ? engine CORS '*' is irrelevant    ?
?            ?                                                               ?
? (3) ENGINE FastAPI  (host=127.0.0.1:8000, HARD-bound [main.py:638])       ?
?      control: jobs table + ThreadPoolExecutor(max_workers=1) drain         ?
?      data: uploads.py chunked upload · /api/compile · /api/highlights      ?
?            ?                                                               ?
? (4) STORAGE = LocalStorage (default)  fetch=identity · put=no-op(src==dst) ?
?            ?                                                               ?
? (5) COMPUTE seam: gemini (cloud, key) · stub (CPU mock) [droplet]          ?
?                   native WASB (GPU host) · auto (owner Win+WSL)            ?
????????????????????????????????????????????????????????????????????????????????
RESERVED SCALE PATH (out of MVP): console "compute target" selector ? storage=s3 (DO
Spaces)
   ? SEPARATE GPU host runs `python -m backend.worker infer --video-uri s3://?` ?
outputs/<vid>/ back
   (needs a NET-NEW orchestration endpoint reflecting outputs into the jobs/poll
contract)
```

4.2 Ingest sources (the headline) ? one menu, one normalized video object

All sources normalize to **"a video the pipeline can read"** ? a local path (MVP) or a
`StorageRef` (scale):

Source	Path today	Status
Local upload	SPA chunks the file ? `max_part_mb` via the existing `/api/upload/init|part|complete` `[verified uploads.py]` ? server path ? `POST /api/process`. (Scale: also `StorageBackend.put` to `s3://`/`gcs://`.)	**wire client only** (endpoints exist)
Bucket URI (`s3://`, `gs://`)	**`POST /api/process` accepts the bucket URI directly** (resolve ? fetch to scratch ? same pipeline ? API DB) `[verified main.py:159,274,283 ? corrected 2026-06-21; was "net-new"]`; the worker CLI also fetches `--video-uri`. So the **engine endpoint is NOT net-new** ? only the SPA call that POSTs the URI + polls `/api/jobs` is. *(Default single-user mode; HOSTED mode w/ `allowed_video_root` correctly rejects a bucket URI 400.)*	**engine: DONE (verified); SPA ingest call: design (= B-P2)**
Google Drive (`gdrive://`)	`parse_uri` recognizes it, but the gdrive backend is an **emulated stub** `[verified ref.py, DECOUPLED_COMPUTE research]`	**stub ? roadmap;** MVP requires a Drive?bucket pre-step; do **not** present Drive as live until the backend is real

4.3 Compute placement matrix (one UI, surfaced from `/api/capabilities`)

Placement	Runs	Needs	Real/mock	Honest caveat
CPU droplet ? gemini (MVP target)	`segmenter=gemini`, in-proc worker	`GEMINI_API_KEY`; network; **no GPU**	**REAL**	Cloud brain: uploads ?1280px chunks to Google ? **must be in consent copy**; **no per-video cost ceiling** today
CPU droplet ? stub	trajectory hybrid + `detector_impl=stub`+`skip_ai_handoff`	nothing	**MOCK**	not real tracking ? UI must label "stub (CPU mock ? not real detection)"; demo/regression only
CPU droplet ? motion	`segmenter=motion`	nothing	REAL, low-quality	only no-key/no-GPU real segmenter; never surface `server/receiver/events` (fabricated)

96 | **GPU host ? native WASB** (in-flight) | `wasb`+`detector_impl=native` ? `NativeWasbRunner` (in-proc torch) | a **GPU host ? DO GPU droplet / GCE GPU VM / Cloud Run+GPU**; `WASB_WEIGHTS` | **REAL**, on-device | partially wired (fusion builds it, trajectory refuses); **owner byte-parity gate**; **Cloud Run/GCE need a containerized worker ? no Dockerfile exists, and WASB's torch-1.11 conda env makes the image non-trivial** |
97 | **Owner Win+WSL ? auto** | `wasb`+`detector_impl=auto` (WSL2+conda) | Windows+WSL2+GPU | REAL (today) | Windows-only transport; the working real-GPU path for owner verification |
98 | **Remote GPU worker via bucket** (reserved) | console CPU + `storage=s3`; separate GPU host runs `backend.worker infer` | bucket creds; a GPU host; a **net-new orchestration endpoint** | REAL (remote) | FastAPI loop and worker CLI are **separate** today; "remote GPU as a setting" is **design** ? selector shown **disabled** in MVP |
99
100   **Provider note:** **Cloud Run + GPU** (scale-to-zero, pay-per-job) is the strongest fit for the engine's own **per-video-billing** economics (`DECOUPLED_COMPUTE` #246) ? gated on the **containerization feasibility check** above.
101
102   ### 4.4 Auth / exposure (non-negotiable invariant)
103   No engine auth, by design; the gate is external. **Invariant (all three must hold):** engine bound to `127.0.0.1` + firewall closes `8000` + edge gate up. Go-live checklist `[HOSTING_PLAN.md:102-108]`: Access/basicauth ON; **rotate the chat-shared `GEMINI_API_KEY`**; set `security.allowed_video_root`; rate-limit; per-user quota.
104
105   ### 4.5 UI operator console (web/ SPA ? superset of RallyPicker)
106   - **S1 Ingest & Process** ? the source menu (§4.2) + a segmenter picker from `/api/capabilities` ? `POST /api/process`.
107   - **S2 Progress** ? 3s poll of the free-text stage (`hashing/validating/segmenting/finalizing`), a **"one job at a time"** notice, and the restart-failure terminal state.
108   - **S3 Rally pick ? reel** ? **embed RallyPicker verbatim** (`PER_RALLY_SELECTION.md`): honor `min_shot_count`; never render motion `server/receiver/events` or a fake confidence meter. **Per-rally is ONE screen of the console.**
109   - **S4 Reels / History** ? needs `GET /api/videos` + `GET /api/reels/{video_id}` for **reload-survival**.
110   - **S5 Settings** ? render `config` + `capabilities`; the compute/storage **target selector shown DISABLED** ("single-box: local") until the bucket-worker bridge lands.
111
112   ### 4.6 Reuse map
113   **Reused (no re-plumb):** the full `/api` loop; the in-process worker; the chunked upload endpoints (un-called); LocalStorage default; the detector seam incl. the landing `NativeWasbRunner`; the **RallyPicker** design + anchors; the `web/` scaffold; the `HOSTING_PLAN` gate; the `SETUP_NEW_MACHINE` runbook. **Net-new:** Caddyfile + systemd units; `GET /api/capabilities`; `GET /api/videos`; `GET /api/reels/{video_id}`; `GET /api/health`; the SPA console screens; ingest-from-bucket wiring; a retention sweep; (reserved) the bucket-worker bridge.
114
115   ## 5. Observability ? logging & monitoring `[required, first-class]`
116   Launch-readiness directive (owner 2026-06-20): **logs and monitoring taken seriously, with a full log-analysis pass across every new code path.** The engine already uses a centralized Python logger `[verified main.py:12-17]` ? carry that discipline into every new path.
117
118   - **Correlation ID end-to-end.** Thread the existing `job_id` (from `/api/process`) **plus** an `upload_id`/`request_id` through ingest ? process ? jobs ? compile ? highlights, and into the worker for the bucket path, so **one CUJ is traceable** across UI ? FastAPI ? worker ? GPU host. Log it on every line.
119   - **Structured logs.** New endpoints/screens emit structured (JSON-able) logs at **entry / result / error** ? not just "it ran". The SPA logs client-side errors + failed fetches with the same correlation id.
120   - **Loud failures, never silent drops.** Carry the engine's ethos (#240 "surface failed chunks loudly") + RallyPicker's `min_shot_count`-warning rule: a partial/empty reel must **never** look complete; every dropped clip/chunk is logged + surfaced in the UI.

23. assistant (2026-06-21T13:53:08.357Z)

This appliance doc is highly relevant. Now let me verify the worker entry point and the server

bind, then do a few more targeted web searches.

24. user (2026-06-21T13:53:10.041Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16      from __future__ import annotations
17
18      import argparse
19      import csv
20      import json
21      import logging
22      import os
23      import sys
24      import tempfile
25      from typing import Any, List
26
27      from backend import storage
28      from backend.compute import ComputeJobSpec, get_compute_backend
29      from backend.config import (
30          StartupCheckError,
31          enforce_startup_checks,
32          load_config,
33          probe_cuda_available,
34      )
35      from backend.infrastructure.database import Database
36      from backend.pipeline.segmenters.base import segmenter_registry
37      from backend.pipeline.validators.base import registry as validator_registry
38      from backend.reporting import emit_indexer_report
39      from backend.results import Failure
40      from backend.utils.hashing import compute_video_id
```

25. user (2026-06-21T13:53:16.428Z)

```
Web search results for query: "pipx install package from private index --pip-args extra-index-
url 2025"

Links: [{"title":"Is there a way to install a package that is located at an index-url that uses
the keyring authentication method? · pypa/pipx · Discussion
#1280","url":"https://github.com/pypa/pipx/discussions/1280"},{"title":"pipx install --index-url
https://test.pypi.org/simple/ --extra-index-url https://pypi.org/simple/ some-package · Issue
#1009 · pypa/pipx","url":"https://github.com/pypa/pipx/issues/1009"},{"title":"How to use pipx
to install namespace package from private repo · Issue #635 ·
pypa/pipx","url":"https://github.com/pypa/pipx/issues/635"},{"title":"pip-args ignored for pip
install that determines package name · Issue #320 ·
pypa/pipx","url":"https://github.com/pipxproject/pipx/issues/320"},{"title":"Configure Paths -
pipx","url":"https://pipx.pypa.io/latest/how-to/configure-paths/"},{"title":"pipx install
<package_spec> | Fig","url":"https://fig.io/manual/pipx/install"},{"title":"pipx ·
```

PyPI","url":"https://pypi.org/project/pipx/0.13.0.1/"},{"title":"Install from artifacts repo (with authentication via artifacts keyring) · Issue #1636 · pypa/pipx","url":"https://github.com/pypa/pipx/issues/1636"}]

Based on the search results, here's information about installing packages from private indexes using pipx with `--pip-args` and `--extra-index-url`:

Using `--pip-args` with `--extra-index-url`

You can use `--pip-args` to use the `--extra-index-url` option. The basic syntax would be:

```bash
pipx install <package-name> --pip-args='--extra-index-url https://pypi.org/simple/'
```

Example with Private Index

A practical example for installing from a private index with keyring authentication would be:
```
pipx install --index-url=https://<gcp-region>-python.pkg.dev/<project>/<registry-name>/simple/ --preinstall=keyrings.google-artifactregistry-auth --pip-args='--extra-index-url https://pypi.org/simple/ --keyring-provider import' <app-name>
```

Alternative: Environment Variables

To point pip at a private index or pass custom options, you can set `PIP_*` environment variables, which pipx forwards to every pip invocation:

```bash
export PIP_INDEX_URL=https://my-private-index.example.com/simple/
export PIP_TRUSTED_HOST=my-private-index.example.com
pipx install my-private-package
```

Per-command pip options can be passed with `--pip-args`: `pipx install my-package --pip-args='--no-cache-dir --trusted-host=my-host'`

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

26. user (2026-06-21T13:53:16.970Z)

Web search results for query: "PyInstaller bundle PyTorch CUDA large size hidden imports problems 2025"

Links: [{"title":"Simplifying Deployment: Using PyInstaller with PyTorch ? codegenes.net","url":"https://www.codegenes.net/blog/pyinstaller-pytorch/"},{"title":"PyInstaller, issue to build a package with CUDA, CuDNN and Tensorflow on Windows 10 · Issue #7175 · pyinstaller/pyinstaller","url":"https://github.com/pyinstaller/pyinstaller/issues/7175"},{"title":"Reducing the Size of PyTorch-Based Executable · pyinstaller · Discussion #8552","url":"https://github.com/orgs/pyinstaller/discussions/8552"},{"title":"Reducing the Size of PyTorch-Based Executable · Issue #8551 · pyinstaller/pyinstaller","url":"https://github.com/pyinstaller/pyinstaller/issues/8551"},{"title":"pyinstaller package size for spacy has significantly increased · Issue #4548 · explosion/spaCy","url":"https://github.com/explosion/spaCy/issues/4548"},{"title":"How to reduce the size of the PyTorch package after using PyInstaller with -D option, resulting in a 3GB size for the torch file? - PyTorch Forums","url":"https://discuss.pytorch.org/t/how-to-reduce-the-size-of-the-pytorch-package-after-using-pyinstaller-with-d-option-resulting-in-a-3gb-size-for-the-torch-file/194822"}]

Based on the search results, here's what I found about PyInstaller bundling PyTorch with CUDA:

Key Challenges

Large Bundle Size: PyTorch with CUDA (torch2.3+cu118) is around 5GB in conda environments, with most of that size being binary extensions and CUDA/cuDNN shared libraries which cannot be split. Even onefile builds with pip-installed torch2.3+cu118 result in around 2.6GB.

Hidden Imports Issues: PyInstaller commands for packaging frameworks like TensorFlow often require multiple hidden imports (e.g., `--hidden-import=skimage.filters.edges --collect-all tensorflow`).

Optimization Solutions

You can optimize package size by removing unnecessary imports from your script, only importing modules and functions you actually need, and using the `--exclude-module` option to exclude certain non-required modules. For example, if not using CUDA, you can exclude CUDA-related modules with `--exclude-module=torch.cuda`.

Realistic Expectations: If others claim to create 200-300MB CUDA-enabled executables, they are likely using CPU-only torch builds or older builds with smaller CUDA libraries.

The fundamental limitation is that CUDA/cuDNN shared libraries take up significant space and cannot be reduced, especially for CUDA-enabled torch builds.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

27. assistant (2026-06-21T13:53:20.611Z)

I have strong material. Let me do a few final targeted searches on uvx with `--with`/`--index`, Windows native dependencies for non-tech users, and the `--torch-backend=auto` flag maturity, then verify the server bind line.

28. user (2026-06-21T13:53:22.036Z)

```
630         _enforce_startup_or_exit(global_config)  # M5: fail fast on an incoherent ACTIVE
profile
631
632         # The CLI --output-dir overrides the configured output directory. It now lives
633         # on the typed model (OutputConfig.output_dir), so every consumer ? including
634         # /api/compile ? reads the same value instead of a write-only runtime hack.
635         global_config.output.output_dir = args.output_dir
636         os.makedirs(global_config.output.output_dir, exist_ok=True)
637
638         db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
639         db_instance = Database(db_path)
640
641         # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
642         # the executor is in-process. Mark them failed so pollers see a terminal state.
643         swept = db_instance.fail_stale_jobs()
644         if swept:
645             logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
646
647         # CLI processing mode (no web UI needed)
648         if args.video_path and not args.server:
649             print(f"[CLI] Processing video file: {args.video_path}")
```

29. user (2026-06-21T13:53:29.234Z)

Web search results for query: "uv standalone installer Windows powershell non-technical user one command install 2025"

Links: [{"title":"Installation | uv","url":"https://docs.astral.sh/uv/getting-started/installation/"},{"title":"How to install uv on Windows | pydevtools","url":"https://pydevtools.com/handbook/how-to/how-to-install-uv-on-windows/"},{"title":"uv · PyPI","url":"https://pypi.org/project/uv/"},{"title":"Installer options | uv","url":"https://docs.astral.sh/uv/reference/installer/"},{"title":"uv Installation Guide: Choosing Between macOS, Linux, Windows, pipx, Homebrew, and WinGet","url":"https://knightli.com/en/2026/05/07/uv-installation-guide/"},{"title":"GitHub - astral-sh/uv: An extremely fast Python package and project manager, written in Rust. · GitHub","url":"https://github.com/astral-sh/uv"},{"title":"uv/docs/getting-started/installation.md at main · astral-sh/uv","url":"https://github.com/astral-sh/uv/blob/main/docs/getting-started/installation.md"},{"title":"How to Install UV package manager on Windows, Linux, Mac, and Docker | by Suhas Vaddhiparthy | Medium","url":"https://vsuhas.medium.com/how-to-install-uv-package-manager-on-windows-linux-mac-and-docker-180cc63c3b18"},{"title":"How to install uv | pydevtools","url":"https://pydevtools.com/handbook/how-to/how-to-install-uv/"}]

Based on the search results, here's what I found about installing uv on Windows using PowerShell for a non-technical user:

One-Command Installation

The official Windows installation command is: `powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"`

Why This Works for Non-Technical Users

The standalone installer is usually a better choice than other methods because it avoids creating a circular dependency (you need Python to install the tool that manages Python).

The installer puts binaries such as uv and uvx under the user directory, and may modify the shell profile so the commands can be used directly in the terminal.

What to Do After Installation

Open a new PowerShell window after running the installer, as the installer updates your user PATH environment variable, but running sessions don't pick up the change.

Alternative Method

If you prefer not to run a remote script directly, Windows package managers like WinGet or Scoop may be a better first choice.

The standalone installer approach is designed to be simple and user-friendly?just paste that single PowerShell command, press Enter, and wait for it to complete.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

30. user (2026-06-21T13:53:31.848Z)

Web search results for query: "uvx --with --index run console script remote package CLI uv tool install --index-url 2025"

Links: [{"title":"Tools | uv","url":"https://docs.astral.sh/uv/concepts/tools/"},{"title":"Using tools | uv","url":"https://docs.astral.sh/uv/guides/tools/"},{"title":"Ultimate guide to uv library in Python","url":"https://deepnote.com/blog/ultimate-guide-to-uv-library-in-python"},{"title":"Packaging Python Command-Line Apps the Modern Way with uv | thisDaveJ","url":"https://thisdavej.com/packaging-python-command-line-apps-the-modern-way-with-uv/"},{"title":"uvx: Run Python CLI Tools in Isolated Environments |

pydevtools","url":"https://pydevtools.com/handbook/reference/uvx/"},{"title":"uv, part 1: running scripts and tools - Paired Ends","url":"https://blog.stephenturner.us/p/uv-part-1-running-scripts-and-tools"},{"title":"uv · PyPI","url":"https://pypi.org/project/uv/"},{"title":"[SOLVED] python uv tool question / Programming & Scripting / Arch Linux Forums","url":"https://bbs.archlinux.org/viewtopic.php?id=306879"}]

Based on the search results, here's what I found regarding the uv CLI tool and its capabilities:

Overview

uvx runs Python command-line tools in temporary isolated environments without installing them permanently and is an alias for uv tool run. These are typically Python packages that provide console scripts entry points, and uv's tool interface lets you run these tools in isolated environments, or install them so they're available as commands globally.

Key Options and Features

--with Option: You can use `uvx --with <extra-package>==<version> <tool-package>` to include additional packages.

--index Option: While the search results don't explicitly detail an `--index` flag, they reference alternative package sources and installation methods.

uv tool install: Tools can be installed with uv tool install, in which case their executables are available on the PATH ? an isolated virtual environment is still used, but it is not removed when the command completes.

--from Option: The --from option can be used to install from alternative sources.

Remote Package Installation

You can host the wheel generated by uv on a web server, allowing users to install and run it directly from a URL using uvx with `uv tool install https://example.com/path/to/wordlookup-0.1.0-py3-none-any.whl`.

For more detailed information about specific command options and their usage, I'd recommend consulting the official uv documentation at docs.astral.sh/uv.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

31. assistant (2026-06-21T13:53:35.304Z)

I have enough to ground the analysis. Let me do two final searches: one to confirm whether `uv tool install` can route torch to a custom index (the critical gap for our app), and one on the `--torch-backend=auto` feature for the worker path.

32. user (2026-06-21T13:53:43.745Z)

Web search results for query: "uv tool install torch index url --default-index --index flag tool run extra index 2025"

Links: [{"title":"Using uv with PyTorch | uv","url":"https://docs.astral.sh/uv/guides/integration/pytorch/"},{"title":"How to Install PyTorch with uv | pydevtools","url":"https://pydevtools.com/handbook/how-to/how-to-install-pytorch-with-uv/"},{"title":"Settings | uv","url":"https://docs.astral.sh/uv/reference/settings/"},{"title":"Managing dependencies | uv","url":"https://docs.astral.sh/uv/concepts/projects/dependencies/"},{"title":"Useful Features of the Python Project Management Tool uv up to v0.5.3 ? Dependencies Edition | by Turing Inc. | Medium","url":"https://medium.com/@turingmotors/5-useful-features-of-the-python-project-management-tool-uv-up-to-v0-5-3-dependencies-edition-9aab2cddceeb"},{"title":"Package indexes |

uv","url":"https://docs.astral.sh/uv/concepts/indexes/"},{"title":"Installing PyTorch: An index URL could not be queried · Issue #8344 · astral-sh/uv","url":"https://github.com/astral-sh/uv/issues/8344"},{"title":"Managing PyTorch CPU/CUDA Versions per Environment with uv Part 2 | MJUN Tech Note","url":"https://mjunya.com/en/posts/2024-10-22-python-uv-pytorch-2/"},{"title":"dependencies - A Developer's Guide to Portable PyTorch Installation with uv and pip - portability","url":"https://iifx.dev/en/articles/460177044/a-developer-s-guide-to-portable-pytorch-installation-with-uv-and-pip"}]

Based on the search results, here's information about using `uv` with PyTorch and custom indexes:

Key Concepts

When an index is marked as `default = true`, it will be moved to the end of the prioritized list and given the lowest priority when resolving packages. Additionally, marking an index as default will disable the PyPI default index.

When defining an index, an explicit flag can be included to indicate that the index should only be used for packages that explicitly specify it in tool.uv.sources. If explicit is not set, other packages may be resolved from the index, if not found elsewhere.

PyTorch Installation with uv

For installing PyTorch specifically, the uv pip interface offers a --torch-backend flag that selects the correct PyTorch index without any pyproject.toml configuration (PyTorch index configuration requires uv 0.5.3 or later).

It's recommended to use explicit = true to ensure that the index is only used for torch, torchvision, and other PyTorch-related packages, as opposed to generic dependencies like jinja2, which should continue to be sourced from the default index (PyPI).

Index Configuration Example

You can configure multiple indexes in pyproject.toml with optional dependency "extras" to choose between different hardware backends (e.g., CPU vs CUDA 12.8), with a conflicts table telling uv these extras are mutually exclusive.

For more details on the specific command-line flags and options you mentioned, you may want to check the official uv documentation at docs.astral.sh/uv/.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

33. user (2026-06-21T13:53:46.694Z)

Web search results for query: ""uv pip install torch" "--torch-backend=auto" detect GPU driver CUDA stable release 2025"

Links: [{"title":"Using uv with PyTorch | uv","url":"https://docs.astral.sh/uv/guides/integration/pytorch/"},{"title":"GitHub - BaoNguyen6742/uv-install-torch: Tutorial to install torch/pytorch with cuda using uv · GitHub","url":"https://github.com/BaoNguyen6742/uv-install-torch"},{"title":"How to Install PyTorch with uv | pydevtools","url":"https://pydevtools.com/handbook/how-to/how-to-install-pytorch-with-uv/"},{"title":"UV Tutorial Part 3: Installing PyTorch with CUDA | by Chandra Kundu | Medium","url":"https://medium.com/@chandrakundu/uv-tutorial-part-3-installing-pytorch-with-cuda-1157bf0d655f"},{"title":"Issues creating a cuda-enabled pytorch environment with UV · Issue #7202 · astral-sh/uv","url":"https://github.com/astral-sh/uv/issues/7202"},{"title":"Streamline CUDA-Accelerated Python Install and Packaging Workflows with Wheel Variants | NVIDIA Technical Blog","url":"https://developer.nvidia.com/blog/streamline-cuda-accelerated-python-install-and-packaging-workflows-with-wheel-variants/"},{"title":"Simplifying PyTorch Environment Setup with uv: A Unified Approach for Cross-Platform and CPU/GPU Support","url":"https://zenn.dev/haru256/articles/6ded722b409d13?locale=en"},{"title":"An

experimental, variant-enabled build of uv","url":"https://astral.sh/blog/wheel-variants"},{"title":"pytorch cuda setup for nvidia gh200 (arm64) | mike bommarito","url":"https://michaelbommarito.com/wiki/programming/languages/python/libraries/pytorch-gh200-arm64/"}]

Based on the search results, here's information about using `uv pip install torch` with GPU driver detection:

GPU Driver Detection with `--torch-backend=auto`

You can use `uv pip install torch --torch-backend=auto` to automatically detect the installed CUDA driver, AMD GPU versions, and Intel GPU presence, then use the most-compatible PyTorch index for all relevant packages (e.g., torch, torchvision, etc.) If no such GPU is found, uv will fall back to the CPU-only index.

Setting `--torch-backend=auto` makes uv detect the available GPU hardware (CUDA driver version, AMD GPU, or Intel GPU) and pick the appropriate backend. If no GPU is detected, it falls back to CPU.

Available Backend Options

Valid values are: auto, cpu, cu118, cu126, cu128, cu130, rocm6, xpu.

Alternative: Using Environment Variables

You can also set the backend via an environment variable: `UV_TORCH_BACKEND=auto uv pip install torch`

Specific CUDA Versions

You can select a specific backend (e.g., CUDA 13.0) with `--torch-backend=cu130`

Important Limitation

`--torch-backend` is only available in the uv pip interface. It does not work with uv lock, uv sync, or uv run. PyTorch index configuration and `--torch-backend` require uv 0.5.3 or later.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

34. assistant (2026-06-21T13:53:51.562Z)

I now have everything I need. I've grounded the engine facts in actual files (pyproject.toml, main.py:638 bind, worker/__main__.py, presets.py, the live appliance doc) and gathered current (2024-2026) best practices on all five channels plus the critical torch `--index-url` handling. Let me compile the structured answer.

A key finding worth flagging: the `--torch-backend=auto` flag is the cleanest answer to the torch problem but is **only available in `uv pip`**, not in `uv tool install`/`uvx`. This is load-bearing for the recommendation.

35. user (2026-06-21T13:55:27.757Z)

Structured output provided successfully

36. assistant (2026-06-21T13:55:40.613Z)

Research complete. I returned the RESEARCH schema via StructuredOutput.

Key grounded findings:

- **The torch `--index-url` problem cuts differently across channels.** It's a real PEP 508

limitation (pip issue #13000, still open). uv's clean answer ? `--torch-backend=auto` and `[tool.uv.index]`/`[tool.uv.sources]` ? only works in the `uv pip`/project interface, **NOT** in `uv tool install`/`uvx` (verified against `docs.astral.sh/uv/guides/integration/pytorch/`). This is load-bearing: you cannot rely on `uv tool install <app>` to also fetch the right torch.

- **uv is the strongest non-tech-Windows fit** because it's the only channel that installs itself in one PowerShell line with auto-PATH AND downloads a correct Python 3.10+ itself ? the India coach needs nothing pre-installed. pipx and pip both require a pre-existing Python and PATH fiddling.

- **The dual-torch reality is decisive.** Per `khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:96`, WASB needs torch 1.11+cu113 in a separate conda env while the engine uses torch 2.6+cu124. No single-venv installer and no single PyInstaller bundle can host both ABIs ? so the distributable must provision the WASB env separately (venv #2 / conda / container). PyInstaller is also impractical here regardless (2.6?5GB, brittle hidden imports for torch/opencv).

- **The existing hatchling wheel needs zero repackaging** to feed any channel; only torch, ffmpeg (a new license-clean dep to vet against the MIT/Apache/BSD guardrail), the WASB second env, and the cross-repo SPA bundle are out-of-band.

Recommendation: lead with uv tool install fronted by a generated `install-indexer.ps1` (bootstrap uv ? install from a private/direct wheel URL ? `uv pip install --torch-backend=auto` into the tool env ? bundle ffmpeg ? first-run GCS weight fetch), backed by a private/direct wheel source, with PyPI+pip as the power-user fallback and a CPU offline bundle for the air-gapped/field persona.

Files grounded: `C:/Users/avidu/Projects/badminton-highlight-indexer/pyproject.toml`, `.../backend/main.py` (bind at :638), `.../backend/worker/__main__.py`, `.../backend/config/presets.py`, and `C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md`.